```
import pandas as pd
import numpy as np
df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
print("==== INITIAL DATA SAMPLE ====")
print(df.head())
print("\nInitial Shape:", df.shape)
print("Missing Values:\n", df.isnull().sum())
df.drop_duplicates(inplace=True)
df.replace(["", " ", "N/A", "na", "null", "-", "--"], np.nan, inplace=True)
df.dropna(subset=['Country', 'Attack Type', 'Year'], inplace=True)
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)
categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
df['Year'] = df['Year'].astype(int)
print("\n==== CLEANED DATA SAMPLE ====")
print(df.head())
print("\nCleaned Shape:", df.shape)
print("Any missing left?", df.isnull().sum().any())
```

```
==== INITIAL DATA SAMPLE ====
   Country  Year        Attack Type     Target Industry  \
0    China  2019           Phishing           Education
1    China  2019         Ransomware              Retail
2    India  2017  Man-in-the-Middle                  IT
3       UK  2024         Ransomware  Telecommunications
4  Germany  2018  Man-in-the-Middle                  IT

   Financial Loss (in Million $)  Number of Affected Users Attack Source  \
0                          80.53                    773169  Hacker Group
1                          62.19                    295961  Hacker Group
2                          38.65                    605895  Hacker Group
3                          41.44                    659320  Nation-state
4                          74.41                    810682       Insider

  Security Vulnerability Type Defense Mechanism Used  \
0          Unpatched Software                    VPN
1          Unpatched Software               Firewall
2              Weak Passwords                    VPN
3          Social Engineering     AI-based Detection
4          Social Engineering                    VPN

   Incident Resolution Time (in Hours)
0                                   63
1                                   71
2                                   20
3                                    7
4                                   68

Initial Shape: (3000, 10)
Missing Values:
 Country                                0
Year                                    0
Attack Type                             0
Target Industry                         0
Financial Loss (in Million $)           0
Number of Affected Users                0
Attack Source                           0
Security Vulnerability Type             0
Defense Mechanism Used                  0
Incident Resolution Time (in Hours)     0
dtype: int64

==== CLEANED DATA SAMPLE ====
   Country  Year        Attack Type     Target Industry  \
0    China  2019           Phishing           Education
1    China  2019         Ransomware              Retail
2    India  2017  Man-in-the-Middle                  IT
3       UK  2024         Ransomware  Telecommunications
4  Germany  2018  Man-in-the-Middle                  IT

   Financial Loss (in Million $)  Number of Affected Users Attack Source  \
0                          80.53                    773169  Hacker Group
1                          62.19                    295961  Hacker Group
2                          38.65                    605895  Hacker Group
3                          41.44                    659320  Nation-state
4                          74.41                    810682       Insider
```

```
import pandas as pd, numpy as np
df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
```

```
df.replace(["", " ", "N/A", "na", "null", "-", "--"], np.nan, inplace=True)
df.dropna(subset=['Country','Attack Type','Year'], inplace=True)
df['Year'] = pd.to_numeric(df['Year'], errors='coerce')
df['Incidents'] = pd.to_numeric(df['Incidents'], errors='coerce').fillna(0) if 'Incidents' in df.columns else 0
df['Sector'] = df['Sector'].fillna('Unknown') if 'Sector' in df.columns else 'Unknown'
df1, df2 = df.iloc[:3], df.iloc[3:6]
concat_df = pd.concat([df1, df2])
appended_df = pd.concat([df1, df2])
df_country = df[['Country', 'Year', 'Incidents']]
df_attack = df[['Country', 'Attack Type']]
merged_df = pd.merge(df_country, df_attack, on='Country', how='inner')
agg = df.groupby('Attack Type')['Incidents'].agg(['sum','mean','count']).reset_index()
pivot = df.pivot_table(index='Country', columns='Attack Type', values='Incidents', aggfunc='sum', fill_value=0)
df['Country_Upper'] = df['Country'].str.upper()
df['Attack_Initial'] = df['Attack Type'].str[0]
df['Year'] = pd.to_datetime(df['Year'], format='%Y', errors='coerce')
ts = df.groupby(df['Year'].dt.year)['Incidents'].sum()
high_incidents = df.query("Incidents > 100")
df['Double_Incidents'] = df.eval("Incidents * 2")
print("✅ CONCAT:\n", concat_df.head(), "\n")
print("✅ APPEND:\n", appended_df.head(), "\n")
print("✅ MERGE (Country + Attack Type):\n", merged_df.head(), "\n")
print("✅ GROUPING (Attack Type - sum/mean/count):\n", agg, "\n")
print("✅ PIVOT TABLE (Country vs Attack Type):\n", pivot.head(), "\n")
print("✅ STRING OPS (Upper Country, Initial Attack):\n", df[['Country', 'Country_Upper', 'Attack Type', 'Attack_Initial']
print("✅ TIME SERIES (Incidents per Year):\n", ts, "\n")
print("✅ QUERY (Incidents > 100):\n", high_incidents[['Country','Attack Type','Incidents']].head(), "\n")
print("✅ EVAL (Double Incidents):\n", df[['Country','Incidents','Double_Incidents']].head())
```

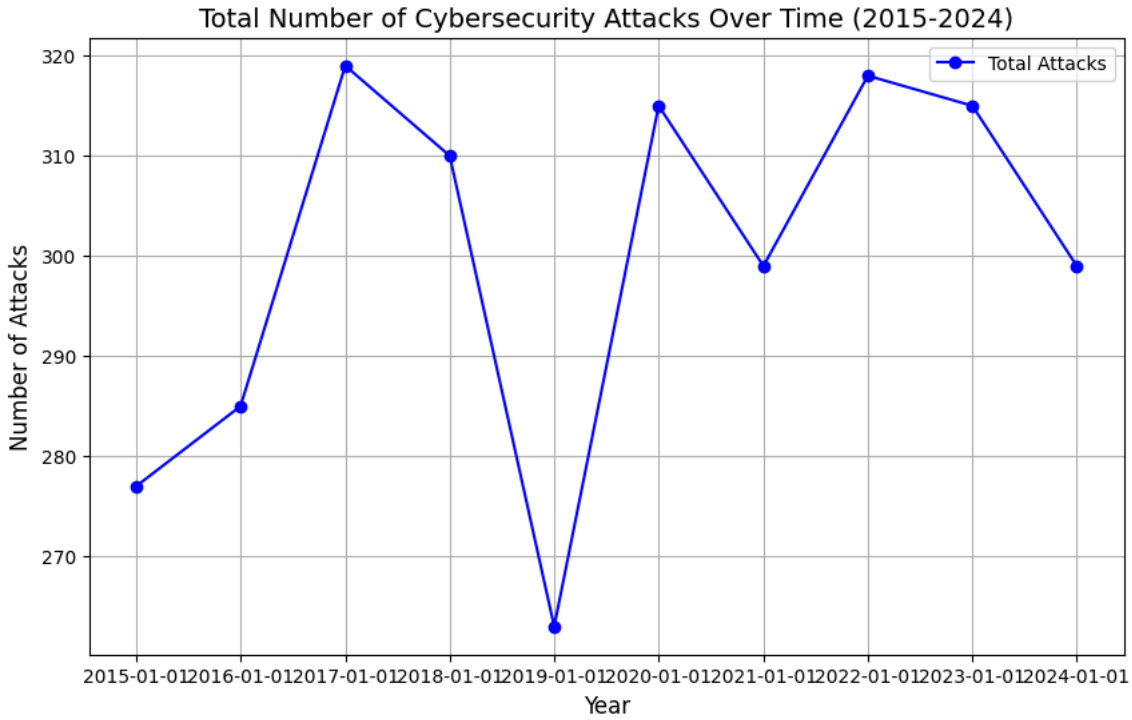
```
✅ CONCAT:
    Country  Year        Attack Type     Target Industry  \
0    China  2019           Phishing           Education
1    China  2019         Ransomware              Retail
2    India  2017  Man-in-the-Middle                  IT
3       UK  2024         Ransomware  Telecommunications
4  Germany  2018  Man-in-the-Middle                  IT

   Financial Loss (in Million $)  Number of Affected Users Attack Source  \
0                          80.53                    773169  Hacker Group
1                          62.19                    295961  Hacker Group
2                          38.65                    605895  Hacker Group
3                          41.44                    659320  Nation-state
4                          74.41                    810682       Insider

  Security Vulnerability Type Defense Mechanism Used  \
0          Unpatched Software                    VPN
1          Unpatched Software                Firewall
2              Weak Passwords                    VPN
3          Social Engineering     AI-based Detection
4          Social Engineering                    VPN

   Incident Resolution Time (in Hours)  Incidents   Sector
0                                   63          0  Unknown
1                                   71          0  Unknown
2                                   20          0  Unknown
3                                    7          0  Unknown
4                                   68          0  Unknown

✅ APPEND:
    Country  Year        Attack Type     Target Industry  \
0    China  2019           Phishing           Education
1    China  2019         Ransomware              Retail
2    India  2017  Man-in-the-Middle                  IT
3       UK  2024         Ransomware  Telecommunications
4  Germany  2018  Man-in-the-Middle                  IT

   Financial Loss (in Million $)  Number of Affected Users Attack Source  \
0                          80.53                    773169  Hacker Group
1                          62.19                    295961  Hacker Group
2                          38.65                    605895  Hacker Group
3                          41.44                    659320  Nation-state
4                          74.41                    810682       Insider

  Security Vulnerability Type Defense Mechanism Used  \
0          Unpatched Software                    VPN
1          Unpatched Software                Firewall
2              Weak Passwords                    VPN
3          Social Engineering     AI-based Detection
4          Social Engineering                    VPN

   Incident Resolution Time (in Hours)  Incidents   Sector
0                                   63          0  Unknown
1                                   71          0  Unknown
2                                   20          0  Unknown
3                                    7          0  Unknown
4                                   68          0  Unknown
```

```
import pandas as pd
import numpy as np
yearly_attack_counts = df.groupby('Year')['Attack Type'].count()
print("Total number of attacks per year:\n", yearly_attack_counts)
attack_type_counts = df.groupby('Attack Type')['Year'].count()
print("\nTotal number of occurrences for each attack type:\n", attack_type_counts)
max_attacks_year = yearly_attack_counts.idxmax()
max_attacks_value = yearly_attack_counts.max()
print(f"\nYear with the maximum number of attacks: {max_attacks_year} ({max_attacks_value} attacks)")
min_attacks_year = yearly_attack_counts.idxmin()
min_attacks_value = yearly_attack_counts.min()
print(f"\nYear with the minimum number of attacks: {min_attacks_year} ({min_attacks_value} attacks)")
most_frequent_attack = attack_type_counts.idxmax()
most_frequent_attack_count = attack_type_counts.max()
print(f"\nMost frequent attack type: {most_frequent_attack} ({most_frequent_attack_count} occurrences)")
mean_attacks = yearly_attack_counts.mean()
print(f"\nAverage number of attacks per year: {mean_attacks:.2f}")
median_attacks = yearly_attack_counts.median()
print(f"\nMedian number of attacks per year: {median_attacks}")
std_dev_attacks = yearly_attack_counts.std()
print(f"\nStandard deviation of attacks per year: {std_dev_attacks:.2f}")
total_attacks = df['Attack Type'].count()
print(f"\nTotal number of attacks in the dataset: {total_attacks}")
min_attack_type = attack_type_counts.idxmin()
min_attack_type_count = attack_type_counts.min()
print(f"\nAttack type with the minimum occurrence: {min_attack_type} ({min_attack_type_count} occurrences)")
unique_countries = df['Country'].nunique()
print(f"\nNumber of unique countries involved in attacks: {unique_countries}")
country_attack_counts = df.groupby('Country')['Attack Type'].count().sort_values(ascending=False)
print("\nNumber of attacks per country:\n", country_attack_counts)
```

```
Total number of attacks per year:
 Year
2015-01-01    277
2016-01-01    285
2017-01-01    319
2018-01-01    310
2019-01-01    263
2020-01-01    315
2021-01-01    299
2022-01-01    318
2023-01-01    315
2024-01-01    299
Name: Attack Type, dtype: int64

Total number of occurrences for each attack type:
 Attack Type
DDoS                 531
Malware              485
Man-in-the-Middle    459
Phishing             529
Ransomware           493
SQL Injection        503
Name: Year, dtype: int64

Year with the maximum number of attacks: 2017-01-01 00:00:00 (319 attacks)

Year with the minimum number of attacks: 2019-01-01 00:00:00 (263 attacks)

Most frequent attack type: DDoS (531 occurrences)

Average number of attacks per year: 300.00

Median number of attacks per year: 304.5

Standard deviation of attacks per year: 19.32

Total number of attacks in the dataset: 3000

Attack type with the minimum occurrence: Man-in-the-Middle (459 occurrences)

Number of unique countries involved in attacks: 10

Number of attacks per country:
 Country
UK           321
Brazil       310
India        308
Japan        305
France       305
Australia    297
Russia       295
Germany      291
USA          287
```

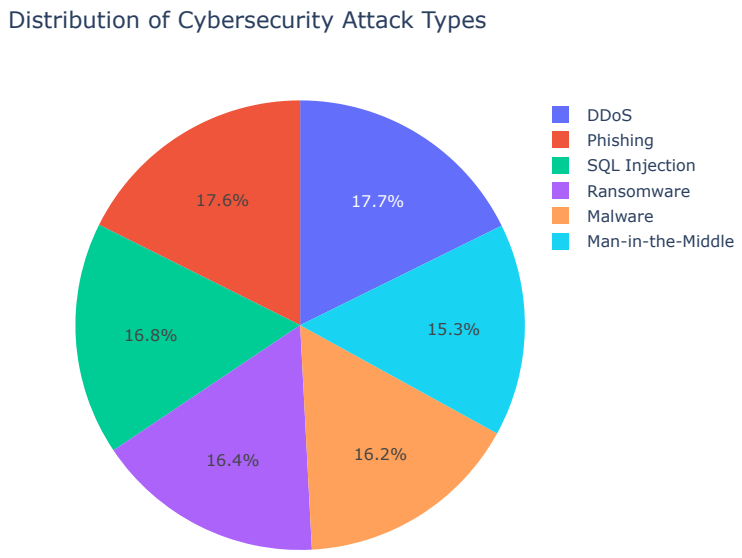
```
China        281
Name: Attack Type, dtype: int64
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
yearly_attack_counts = df.groupby('Year')['Attack Type'].count()
plt.figure(figsize=(10, 6))
plt.plot(yearly_attack_counts.index, yearly_attack_counts.values, marker='o', color='b', label='Total Attacks')
plt.title('Total Number of Cybersecurity Attacks Over Time (2015-2024)', fontsize=14)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Attacks', fontsize=12)
plt.grid(True)
plt.xticks(yearly_attack_counts.index)
plt.legend()
plt.show()
attack_type_frequency = df['Attack Type'].value_counts()
print("Top 5 Most Frequent Attack Types:")
print(attack_type_frequency.head())
mean_attacks = yearly_attack_counts.mean()
median_attacks = yearly_attack_counts.median()

print(f"\nAverage Number of Attacks per Year: {mean_attacks:.2f}")
print(f"Median Number of Attacks per Year: {median_attacks}")
```

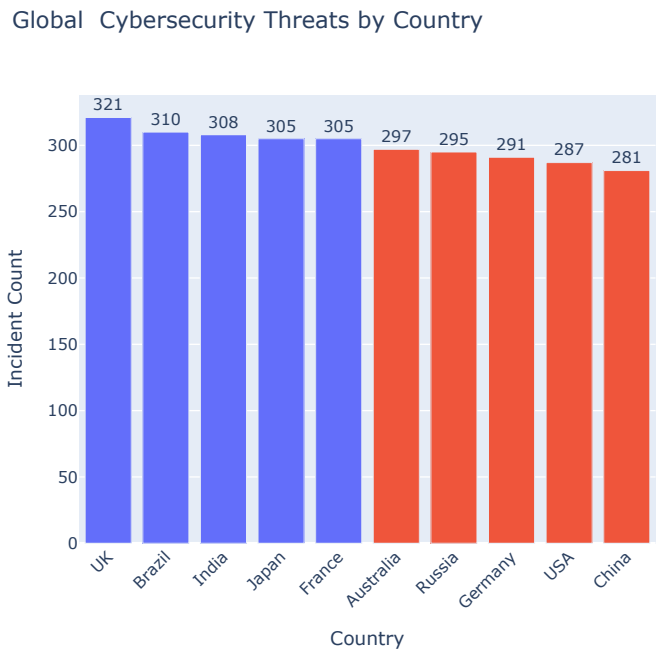



```
Top 5 Most Frequent Attack Types:
Attack Type
DDoS             531
Phishing         529
SQL Injection    503
Ransomware       493
Malware          485
Name: count, dtype: int64

Average Number of Attacks per Year: 300.00
Median Number of Attacks per Year: 304.5
```

```
import plotly.express as px
attack_type_counts = df['Attack Type'].value_counts().reset_index()
attack_type_counts.columns = ['Attack Type', 'Count']
fig = px.pie(attack_type_counts,
             names='Attack Type',
             values='Count',
             title='Distribution of Cybersecurity Attack Types',
             hover_data={'Attack Type': True, 'Count': True, 'Count': True, 'Attack Type': True})

fig.show()
```

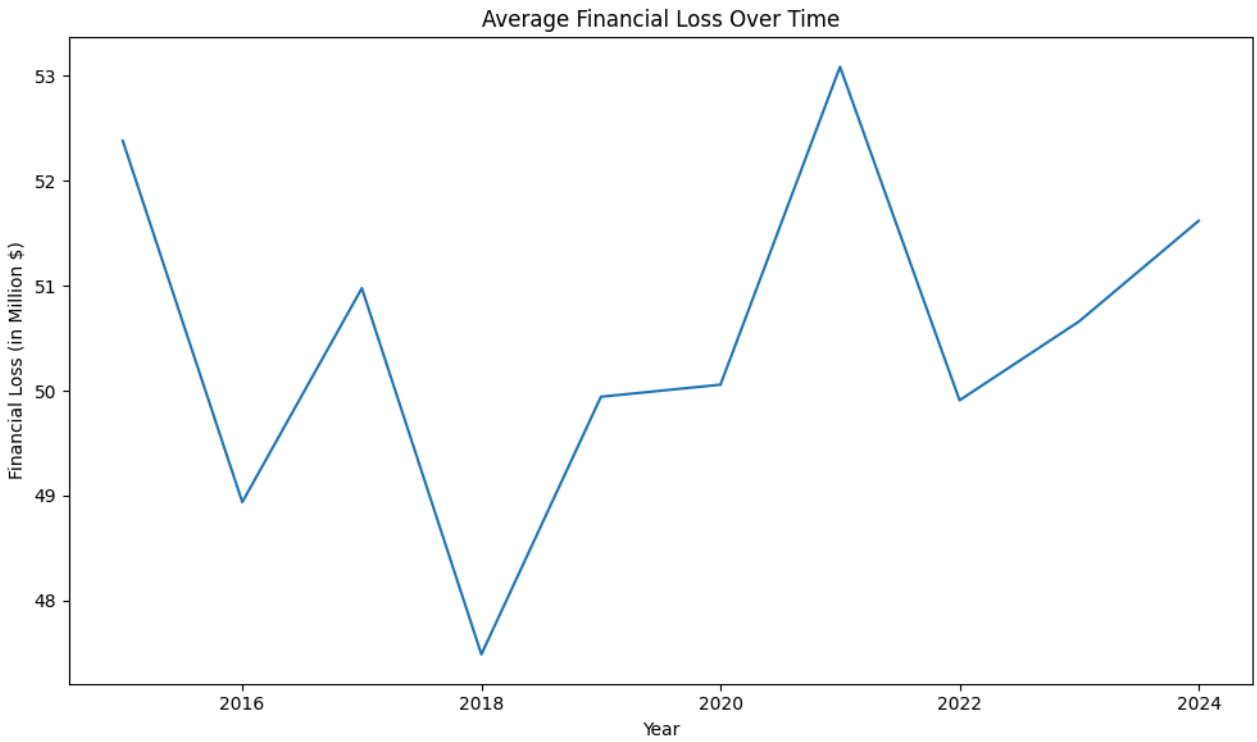



```
import pandas as pd
import plotly.express as px
df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
country_counts = df['Country'].value_counts().reset_index()
country_counts.columns = ['Country', 'Incident Count']
def get_color(count):
    if count > 300:
        return 'red'
    elif count > 200:
        return 'orange'
    elif count > 100:
        return 'yellowgreen'
    else:
        return 'lightblue'

country_counts['Color'] = country_counts['Incident Count'].apply(get_color)
fig = px.bar(
    country_counts,
    x='Country',
    y='Incident Count',
    color='Color',
    text='Incident Count',
    title="Global  Cybersecurity Threats by Country"
)

fig.update_traces(textposition='outside', hovertemplate='Country: %{x}<br>Incidents: %{y}')
fig.update_layout(xaxis_tickangle=-45, showlegend=False)
fig.show()
```



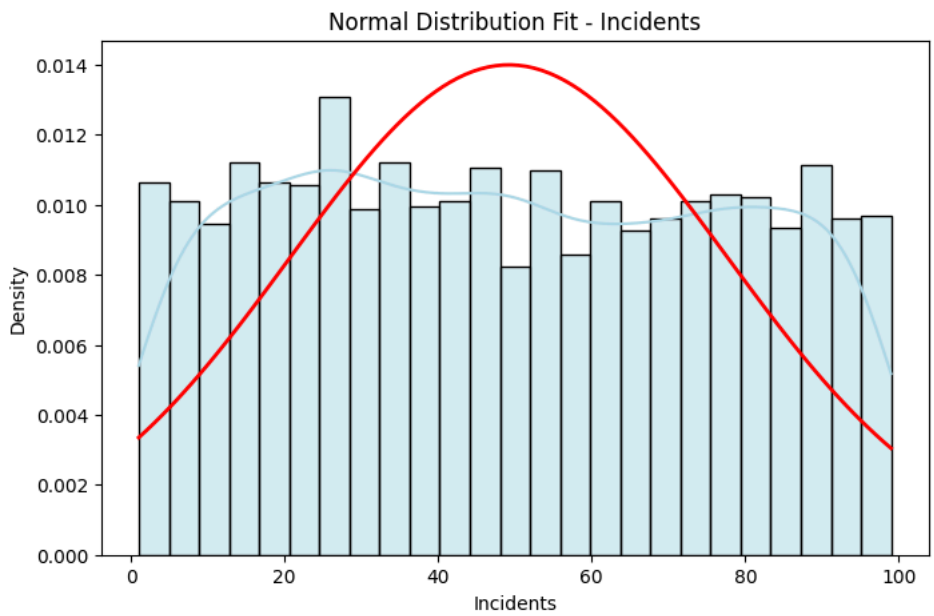

```
import seaborn as sns
plt.figure(figsize=(10, 6))
sns.lineplot(data=df, x='Year', y='Financial Loss (in Million $)', estimator='mean', ci=None)
plt.title('Average Financial Loss Over Time')
plt.ylabel('Financial Loss (in Million $)')
plt.xlabel('Year')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_15755/522448868.py:3: FutureWarning:


The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.
```

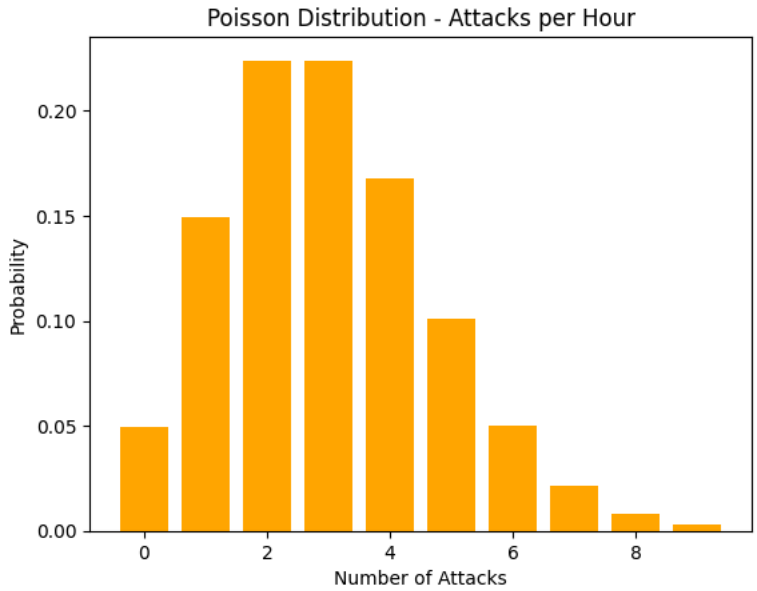



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

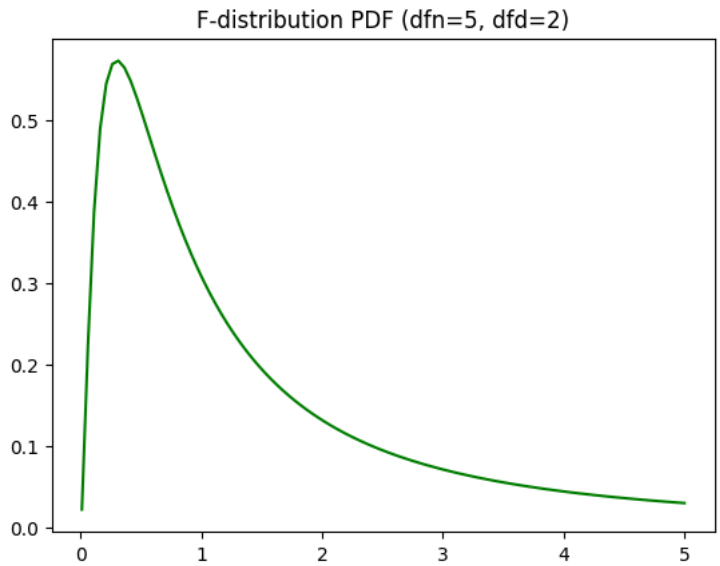
```
import seaborn as sns
from scipy import stats
from scipy.stats import norm, poisson, bernoulli, ttest_ind, chi2_contingency, f_oneway, zscore
import warnings
warnings.filterwarnings("ignore")
df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
df.replace(["", " ", "N/A", "na", "null", "-", "--"], np.nan, inplace=True)
df.drop_duplicates(inplace=True)
df.dropna(subset=['Country', 'Attack Type', 'Year'], inplace=True)
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)

categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

df['Year'] = df['Year'].astype(int)
if 'Incidents' not in df.columns:
    df['Incidents'] = np.random.randint(1, 100, size=len(df))
plt.figure(figsize=(8, 5))
sns.histplot(df['Incidents'], kde=True, stat="density", color="lightblue", bins=25)
x = np.linspace(df['Incidents'].min(), df['Incidents'].max(), 100)
p = norm.pdf(x, df['Incidents'].mean(), df['Incidents'].std())
plt.plot(x, p, 'r', linewidth=2)
plt.title("Normal Distribution Fit - Incidents")
plt.show()
```




```
x = np.arange(0, 10)
mu = 3
plt.bar(x, poisson.pmf(x, mu), color="orange")
plt.title("Poisson Distribution - Attacks per Hour")
plt.xlabel("Number of Attacks")
plt.ylabel("Probability")
plt.show()
```




```
x = np.linspace(0.01, 5, 100)
y = stats.f.pdf(x, dfn=5, dfd=2)
plt.plot(x, y, 'g-')
plt.title("F-distribution PDF (dfn=5, dfd=2)")
plt.show()
```

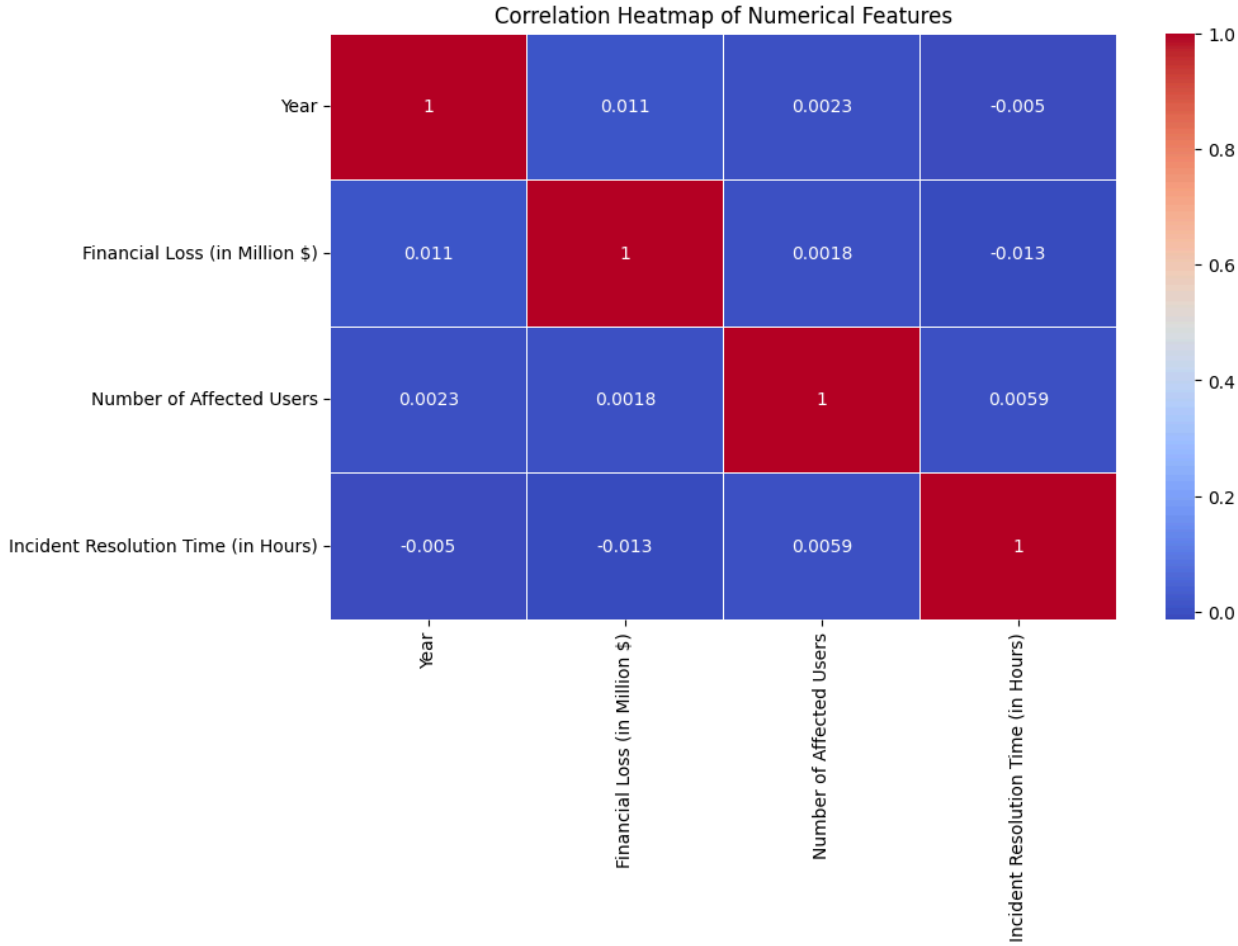



```
if 'Severity' not in df.columns:
    df['Severity'] = np.random.uniform(1, 10, size=len(df))
print("Correlation (Incidents vs Severity):", df['Incidents'].corr(df['Severity']))
```

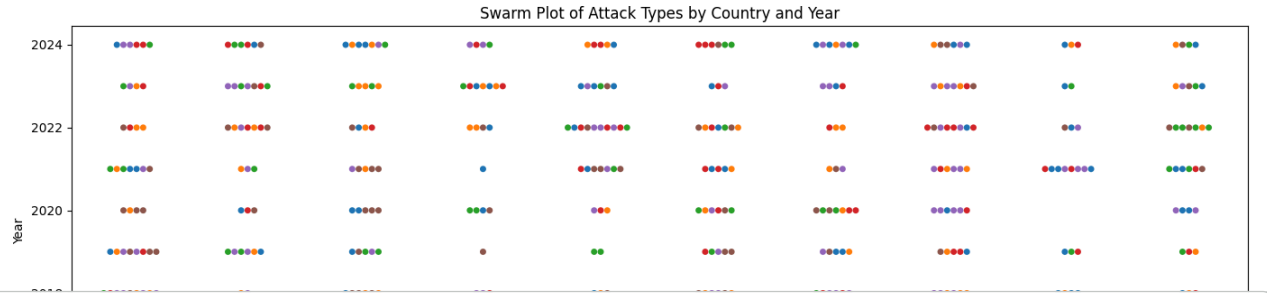
```
Correlation (Incidents vs Severity): 0.008803140422716578
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
df.replace(["", " ", "N/A", "na", "null", "-", "--"], np.nan, inplace=True)
df.drop_duplicates(inplace=True)
df.dropna(subset=['Country', 'Attack Type', 'Year'], inplace=True)

numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)

categorical_cols = df.select_dtypes(include='object').columns
```

```
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

df['Year'] = df['Year'].astype(int)

plt.figure(figsize=(10, 6))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features")
plt.show()

if 'Sector' in df.columns:
    plt.figure(figsize=(12, 6))
    sns.violinplot(data=df, x='Sector', y='Year', hue='Attack Type', split=True)
    plt.title("Distribution of Attacks by Sector and Year")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
else:
    print("⚠️ 'Sector' column not found in the DataFrame. Skipping violin plot.")
df_sample = df.sample(500, random_state=42)
plt.figure(figsize=(14, 6))
sns.swarmplot(data=df_sample, x='Country', y='Year', hue='Attack Type')
plt.title("Swarm Plot of Attack Types by Country and Year")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
for attack in df['Attack Type'].unique():
    subset = df[df['Attack Type'] == attack]
    sns.kdeplot(subset['Year'], label=attack, fill=True)
plt.title("KDE Plot of Attack Trends Over Years")
plt.legend()
plt.show()
if 'Incidents' in df.columns:
    plt.figure(figsize=(10, 6))
    sns.boxplot(data=df, x='Sector', y='Incidents', palette='Set2')
    plt.title("Box Plot of Incident Count by Sector")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
selected_cols = numeric_cols[:3] if len(numeric_cols) > 3 else numeric_cols
sns.pairplot(df[selected_cols.tolist() + ['Attack Type']], hue='Attack Type', palette='husl')
plt.suptitle("Pairplot of Selected Numeric Features", y=1.02)
plt.show()
```




```
⚠️ 'Sector' column not found in the DataFrame. Skipping violin plot.
```




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings
warnings.filterwarnings("ignore")
```

```
df = pd.read_csv("Global_Cybersecurity_Threats_2015-2024.csv")
print("Initial Shape:", df.shape)
```

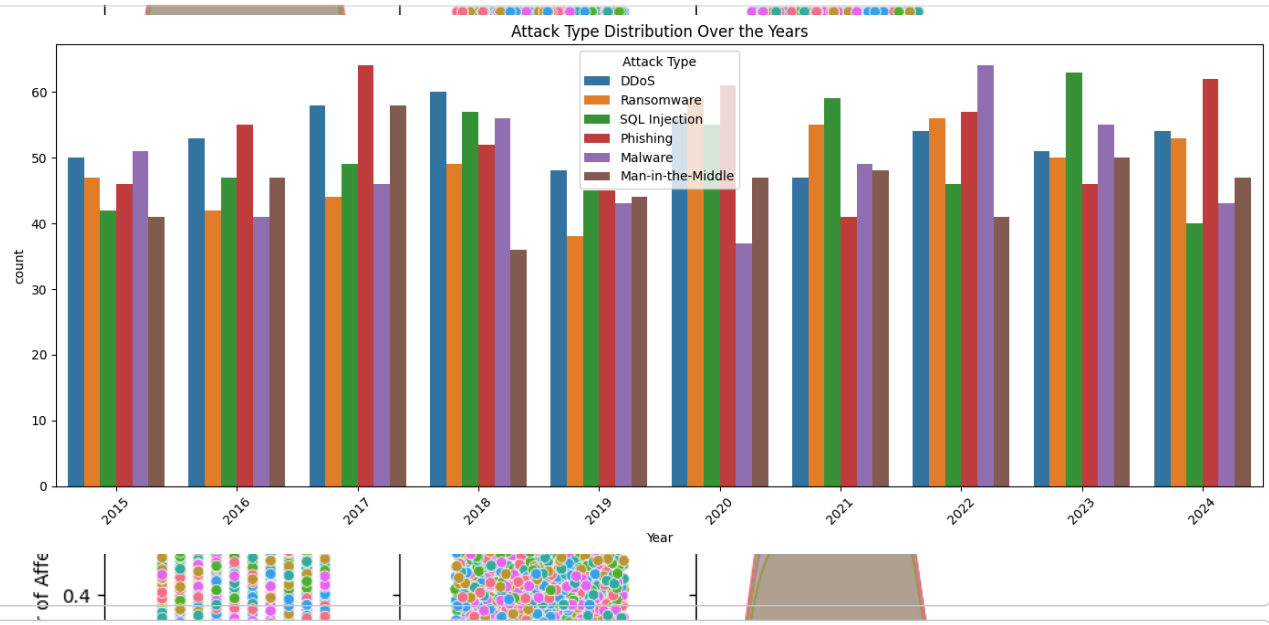
```
Initial Shape: (3000, 10)
```

```
df.replace(["", " ", "N/A", "na", "null", "-", "--"], np.nan, inplace=True)
df.drop_duplicates(inplace=True)
df.dropna(subset=['Country', 'Attack Type', 'Year'], inplace=True)

numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)
```

```
categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

df['Year'] = df['Year'].astype(int)
```

Pairplot of Selected Numeric Features

```
plt.figure(figsize=(14, 6))
sns.countplot(data=df, x='Year', hue='Attack Type')
plt.title("Attack Type Distribution Over the Years")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
label_encoders = {}
for col in ['Country', 'Sector', 'Attack Type']:
    if col in df.columns:
        le = LabelEncoder()
        df[col + '_Encoded'] = le.fit_transform(df[col])
        label_encoders[col] = le
```

```
features = ['Country_Encoded', 'Year']
if 'Sector_Encoded' in df.columns:
    features.append('Sector_Encoded')
if len(numeric_cols) > 0:
    features += list(numeric_cols)

X = df[features]
y = df['Attack Type_Encoded']
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.25, random_state=42)
```

```
rf = RandomForestClassifier(n_estimators=150, random_state=42)
rf.fit(X_train, y_train)
rf_preds = rf.predict(X_test)

print("🎯 Random Forest Accuracy:", accuracy_score(y_test, rf_preds))
print("\nClassification Report:\n", classification_report(y_test, rf_preds))
```

```
🎯 Random Forest Accuracy: 0.152

Classification Report:
               precision    recall  f1-score   support
```